

Declaración de IAG – Grupo 3 – Reto #1

- Juan Andrés Ospina Ramirez - J.ospinar2@uniandes.edu.co – 202614830
- Jose Alexander Santos Cobos - j.santosc2@uniandes.edu.co – 202613656
- Juan Esteban Lesmes Chaves - je.lesmesc1@uniandes.edu.co – 202614029

Descripción de la parte en donde la IAG fue usada

Para el reto 1 de la clase Estructuras de Datos y Algoritmos, se hizo uso de inteligencia artificial generativa para la creación de funciones auxiliares relacionadas con la carga de datos provenientes de archivos CSV. Dentro de las creadas, podemos ver funciones de conversión, lectura de datos y resumen de datos que posteriormente se verán implementados en el apartado de visualización por medio de prints. De igual forma, las funciones fueron creadas para permitir la utilización de distintos archivos para realizar distintas pruebas con volúmenes de datos diferentes. Estas se implementaron tanto en el archivo *logic.py* como en el *view.py* ya que la carga de datos debía ser implementada tanto en la lógica del reto como para su respectiva visualización.

Justificación del uso

Se hizo uso de la inteligencia artificial generativa (IAG) ya que, para el requerimiento de la carga de datos, conocíamos cómo realizar una carga de datos convencional haciendo uso de la librería Pandas, sin embargo, no teníamos claro cómo hacerla usando Python nativamente sin recurrir a librerías. Después de haber recurrido a laboratorios previos al reto, no encontramos manera alguna de implementarla y al ser fundamental para que el código pudiera funcionar, recurrimos a la IAG.

Evidencia del uso

```
41 def convertir_fila(fila):
42     """
43     Convierte una fila del CSV (dict de strings) en un pedido con tipos correctos
44     """
45     order = {
46         "Order_ID": fila.get("Order_ID") or "Unknown",
47         "Product": fila.get("Product") or "Unknown",
48         "Country": fila.get("Country") or "Unknown",
49         "Channel": fila.get("Channel") or "Unknown",
50         "Order_Date": fila.get("Order_Date") or "Unknown",
51         "Discount_Pct": to_float(fila.get("Discount_Pct")),
52         "Price_per_Box": to_float(fila.get("Price_per_Box")),
53         "Marketing_Spend": to_float(fila.get("Marketing_Spend")),
54         "Boxes_Shipped": to_int(fila.get("Boxes_Shipped")),
55         "Amount": to_float(fila.get("Amount")),
56     }
57
58     return order
59
60 def load_data(catalog, filename):
61     """
62     Carga los datos del reto
63     """
64
65     ruta = os.path.join("Data", filename)
66
67     with open(ruta, encoding="utf-8-sig") as archivo:
68         lector = csv.DictReader(archivo)
69         for fila in lector:
70             pedido = convertir_fila(fila)
71             lt.add_last(catalog["ordenes"], pedido)
72
73     return catalog
74
```

```

75 def resumen_carga(catalog):
76     """
77     Calcula las estadísticas que pide la Parte 2 sobre los datos ya cargados
78     """
79     pedidos = catalog["ordenes"]
80     total = lt.size(pedidos)
81
82     menor = lt.get_element(pedidos, 0)
83     mayor = lt.get_element(pedidos, 0)
84
85     for i in range(total):
86         actual = lt.get_element(pedidos, i)
87
88         if actual["Amount"] > mayor["Amount"] or (actual["Amount"] == mayor["Amount"] and actual["Price_per_Box"] < mayor["Price_per_Box"]):
89             mayor = actual
90
91         if actual["Amount"] < menor["Amount"] or (actual["Amount"] == menor["Amount"] and actual["Price_per_Box"] < menor["Price_per_Box"]):
92             menor = actual
93
94     primeros_5 = []
95     cantidad_primeros = min(5, total)
96     for i in range(cantidad_primeros):
97         primeros_5.append(lt.get_element(pedidos, i))
98
99     ultimos_5 = []
100     inicio_ultimos = max(0, total - 5)
101     for i in range(inicio_ultimos, total):
102         ultimos_5.append(lt.get_element(pedidos, i))
103
104     return {
105         "total": total,
106         "menor": menor,
107         "mayor": mayor,
108         "primeros_5": primeros_5,
109         "ultimos_5": ultimos_5,
110     }

```

```

33 def load_data(control):
34     """
35     Carga los datos
36     """
37     print("\n¿Qué archivo desea cargar?\n")
38     print("1- 100 elementos (pruebas rápidas)")
39     print("2- 20%")
40     print("3- 40%")
41     print("4- 60%")
42     print("5- 80%")
43     print("6- 100% (Definitivo)")
44     opcion = input("Seleccione una opción: ")
45     filename = archivos.get(opcion, archivos["1"])
46
47     start_time = logic.get_time()
48     logic.load_data(control, filename)
49     end_time = logic.get_time()
50     tiempo = logic.delta_time(start_time, end_time)
51
52     resumen = logic.resumen_carga(control)
53     print_resumen_datos(tiempo, resumen)
54
55 def print_resumen_datos(tiempo, resumen):
56
57     print("\nTiempo de carga: {:.2f} ms".format(tiempo))
58     print("Total de pedidos cargados: {}".format(resumen["total"]))
59
60     print("\nPedido de menor cantidad:")
61     print_pedido(resumen["menor"])
62
63     print("\nPedido de mayor cantidad:")
64     print_pedido(resumen["mayor"])
65
66     print("\nPrimeros 5 registros:")
67     for p in resumen["primeros_5"]:
68         print_pedido(p)
69
70     print("\nÚltimos 5 registros:")
71     for p in resumen["ultimos_5"]:
72         print_pedido(p)

```

```

33 def load_data(control):
34     """
35     Carga los datos
36     """
37     print("\n¿Qué archivo desea cargar?\n")
38     print("1- 100 elementos (pruebas rápidas)")
39     print("2- 20%")
40     print("3- 40%")
41     print("4- 60%")
42     print("5- 80%")
43     print("6- 100% (Definitivo)")
44     opcion = input("Seleccione una opción: ")
45     filename = archivos.get(opcion, archivos["1"])
46
47     start_time = logic.get_time()
48     logic.load_data(control, filename)
49     end_time = logic.get_time()
50     tiempo = logic.delta_time(start_time, end_time)
51
52     resumen = logic.resumen_carga(control)
53     print_resumen_datos(tiempo, resumen)
54
55 def print_resumen_datos(tiempo, resumen):
56
57     print("\nTiempo de carga: {0:.2f} ms".format(tiempo))
58     print("Total de pedidos cargados: {0}".format(resumen["total"]))
59
60     print("\nPedido de menor cantidad:")
61     print_pedido(resumen["menor"])
62
63     print("\nPedido de mayor cantidad:")
64     print_pedido(resumen["mayor"])
65
66     print("\nPrimeros 5 registros:")
67     for p in resumen["primeros_5"]:
68         print_pedido(p)
69
70     print("\nÚltimos 5 registros:")
71     for p in resumen["ultimos_5"]:
72         print_pedido(p)
73

```

view.py

```

def print_pedido(p):
    print(" {0} | {1} | {2} | {3} | {4} | ${5:.2f} | ${6:.2f}".format(p["Order_ID"], p["Product"], p["Country"],
p["Channel"],p["Order_Date"], p["Price_per_Box"], p["Amount"]))

```



De esta manera se ve el CSV, cómo los cargo todos o cómo hago para cargar uno diferente y cuando quiera hacer pruebas con uno y el otro

Pensó durante 4s Working out how to load and switch CSV sample files.

Bien, ya vi el formato del CSV — coincide exactamente con la Tabla 1 del enunciado. Ahora, para escribirte el código de carga necesito ver cómo están armadas las estructuras de datos en tu plantilla (los nombres de las clases y sus métodos son distintos según la plantilla del curso).

¿Me puedes mandar una captura de:

1. La estructura de carpetas completa del proyecto (el árbol de archivos, como el panel izquierdo de tu editor).
2. El contenido del archivo donde está definida tu **ArrayList** (o como se llame en tu plantilla) — solo para ver los nombres de sus métodos (`add`, `append`, `get`, `size`, etc.).

Reflexión breve del aprendizaje obtenido

Con ayuda de la IAG, hemos entendido cómo realizar una carga de datos no solo para archivos individuales sino con la posibilidad de ir alternando para realizar distintos tipos de pruebas. En adición, muchas partes del código generado no las hubiésemos podido realizar sin apoyarnos en IAG, ya que aún no conocemos bien cómo funciona *utf-8* y bueno, en este caso, tuvimos que implementar *utf-8-sig* para que el código pudiese correr apropiadamente. Esto nos deja motivación para seguir informándonos más acerca de funciones nativas de Python y recursos que podemos descubrir para hacer en próximas ocasiones cargas de datos óptimas y hasta mejor implementadas.